Exercise on BVA and Code Coverage

Week 4: Specification-based and Structural Testing

Below are example solutions to the exercises. While these address the requirements, you are encouraged to explore alternative approaches and solutions. Your creativity and problem-solving strategies are valued :) (Solutions for the optional exercises are not provided)

Part 1: Boundary Value Analysis

Specification: A function calculateGrade evaluates a student's performance based on a numeric score. The function has the following rules:
If the score is less than 0 or greater than 100, the function throws an exception.
If the score is between 0 and 49 (inclusive), the function returns "Fail."
If the score is between 50 and 74 (inclusive), the function returns "Pass."
If the score is between 75 and 89 (inclusive), the function returns "Merit."
If the score is between 90 and 100 (inclusive), the function returns "Distinction."

Perform Boundary Value Analysis (BVA) on this specification

Q1.1: Identify equivalence partitions

Partition name	Input description	Expected behaviour
Out-of-range score	Scores less than 0 or greater than 100.	Throw an exception
"Fail" range	Valid score between 0 and 49	Return "Fail"
"Pass" range	Valid score between 50 and 74	Return "Pass"
"Merit" range	Valid score between 75 and 89	Return "Merit"
"Distinction" range	Valid score between 90 and 100	Return "Distinction"

Q1.2: Find boundary test cases (note: boundaries are pairs of input)

Partitions	Input Pair
Out-of-range score & "Fail" range	i1 = -1 and i2 = 0

“Fail” range & “Pass” range	i1 = 49 and i2 = 50
“Pass” range & “Merit” range	i1 = 74 and i2 = 75
“Merit” range & “Distinction” range	i1 = 89 and i2 = 90
“Distinction” range & Out-of-range score	i1 = 100 and i2 = 101

Q1.3: Calculate program derivatives (PD) for those test cases. Are there any problems with the current selection of distance calculation?

Tip: You can use different formulas for calculating both input and output distances. The goal is to check how “close” or “similar” the values are. The selection of formulas depends on different factors (e.g., type of values, resources, etc.). In this example, you can use the following distance functions:

- `Input distance: Euclidean distance =`
  - `d(i1, i2) = abs( i2 - i1 )`
- `Output distance: Diff in String length:`
  - `d(out1,out2) = abs ( out1.length() - out2.length() )`

PD = output distance / input distance

i1	**i2**	**output1**	**output2**	**input dist**	**output dist**	**PD**
-1	0	“InvalidInput”	“Fail”	1	8	8
49	50	“Fail”	“Pass”	1	0	0
74	75	“Pass”	“Merit”	1	1	1
89	90	“Merit”	“Distinction”	1	6	6
100	101	“Distinction”	“InvalidInput”	1	1	1

Depending on your specific measurement goals and available resources, different distance functions can be chosen. However, there is an issue with the current selection for output distance. For example, string length distance is a fast and simple approximation for comparing strings. Yet, consider the case of “Fail” and “Pass”: their string length difference is 0, resulting in a program derivative value of 0. Despite being distinct outputs, the chosen distance metric fails to reflect this difference.

Specification 2: A function `parseTermNames(int[] monthIndex)` parses an array of months of the years (as integers) into the corresponding academic period in a lecture year. There are three academic periods:
(i) The "Spring term" covers the months between January (1) until May (5);
(ii) The "Summer break" happens between June (6) and August (8);
(iii) The "Autumn term" includes the months between September (9) and December (12);

The program can throw two types of Exceptions: the first covers cases where the array of month indices is empty or null (`"Error. Month array cannot be empty or null"`), whereas the second cover cases where there is any invalid month index in the input array (`"Invalid month. Month must be between 1:12"`). Java does not allow null values inside the array.

For example:

- `parseTermNames([1, 7, 9]) returns ["Spring term", "Summer break", "Autumn term"]`
- `parseTermNames([]) returns "Error. Month array cannot be empty or null"`
- `parseTermNames([0]) returns "Invalid month. Month must be between 1:12"`

Q2.1 Identify equivalence partitions

Partition name	**Input description**	**Expected behaviour**
P1: Invalid input (empty or null)	monthIndex array is null or empty	Should throw an exception indicating that the array of integers is not valid.
P2: Invalid input (month outside of range)	monthIndex array contains at least one element that is outside of range 1:12	Should throw an exception for the invalid month value.
P3: Valid input (all values are the same)	monthIndex array contains more than one element; all elements are the same	Return an array with all values in the same term.
P4: Valid input (values from all terms)	monthIndex array contains at least one element for each of the terms	All three different terms are expected to be seen in the output array.

P5: Valid input (one element matching Spring term)	monthIndex contains one element only, which is in the range 1:5	Should return an array with one element: “Spring term”
P6: Valid input (one element matching Summer break)	monthIndex contains one element only, which is in the range 6:8	Should return an array with one element: “Summer break”
P7: Valid input (one element matching Autumn term)	monthIndex contains one element only, which is in the range 9:12	Should return an array with one element: “Autumn term”

Q2.2: Find boundary test cases. Identify the validity group for each boundary pair. Find at least 2 boundaries per validity group. (note: validity groups are VV, VE, EE)

Note: there are other input pair examples that are boundary test cases between the partitions in Q2.1. Try to prioritize those which have “closer” inputs and “father” from each other outputs.

Partitions	**Input Pair**	**Expected Output**	**Validity group**
P1 and P5	i1 = [] and i2 = [1]	o1 = “Error…” o2 = [“Spring term”]	VE
P2 and P3	i1 = [0, 6, 6] and i2 = [6, 6, 6]	o1 = “Invalid month…” o2 = [“Summer break", “Summer break", “Summer break”]	VE
P1 and P3	i1 = null and i2 = [0]	o1 = “Error…” o2 = “Invalid month…”	EE
P1 and P3	i1 = [] and i2 = [13]	o1 = “Error…” o2 = “Invalid month…”	EE
P6 and P7	i1 = [8] and i2 = [9]	o1 = [“Summer break”] o2 = [“Autumn term”]	VV
P3 and P4	i1 = [1,1,1] and i2=[1, 6, 9]	o1 = [“Spring term”, “Spring term”, “Spring term”] o2 = [“Spring term”, “Summer break”, “Autumn term”]	VV

Part 2: Structural Testing and Test Adequacy Criteria

Context:
You have been recently contracted to evaluate and enhance the test suite of a software product in development. The software product is a cinema management system, where a cinema has various halls where movies are screened.

Your contractor has sent you the source code `cinema_manager-master.zip` (or link below), which you can download at **Canvas > Files > Other material > Exercise Files.** However, they have forgotten to include the specification of the system to be developed! For this reason, you will have to base any new tests on white-box techniques and your own understanding of the System Under Test (**SUT**).

Instructions: (Note: Make sure you are using Python 3.8 or higher)

The learning goal of using this small Python project is to practise assessing the test adequacy of a test suite and then to perform structural testing. Start by downloading the software product. Follow the instructions in README.md in order to install all dependencies required during this exercise.

Q2.1 Before you can start evaluating the test suite you should **familiarise with the code provided**. Look at the modules and their respective tests in the following order:

1. Movie
2. Hall
3. Cinema

Without using code coverage or mutation testing, (i) **write down your initial impression of the testing suite?** (ii) What is your **"rough" estimate** on the code coverage? (iii) Do you believe there are faults that the test suite would miss? **Justify** your answers.

(i) Write down your initial impression of the testing suite?

The initial impression of the testing suite is that it is incomplete and lacks sufficient coverage across various functions. Below are just a few examples, there are additional uncovered branches and scenarios throughout the test suite that are not addressed.

In the Movie module, for instance, the get_ticket_price function depends on two factors: in_high_demand and days_between_datetimes. However, the current tests only cover a single scenario where the ticket is purchased 8 days in advance, for both high and low demand. Critical scenarios, such as purchases made less than 5 days or more than 30 days in advance, remain untested, leaving significant branches unexplored.

Similarly, the get_number_of_breaks function, which calculates breaks based on movie duration and genre, is tested only with a 180-minute duration and the Thriller genre. This limited coverage is insufficient, as different durations and genres could impact the calculations, and these cases are currently missing from the test suite.

The get_break_times function, which includes a loop, is tested in just one scenario. Ideally, the suite should include tests where the loop iterates 0 times, 1 time, and multiple times to ensure proper functionality across all conditions, but these cases are not included.

In the Hall module, the functions test_plan_movie and test_book_tickets fail to test scenarios where a ValueError should be raised, leaving the error-handling mechanisms unverified.

(ii) What is your “rough” estimate on the code coverage in percentage?

Note: For this exercise coverage refers to:
`Coverage=(total-missed)/total,`
`missed=(missedBranches+missedLines)`
`total=(totalBranches+totalLines).`
You do not need to compute this value. It’s provided only to clarify what we mean by “coverage” in this context.

Based on the limited scenarios tested in the suite and the absence of branch coverage across various functions, a rough estimate of the code coverage would likely be below 70%. This estimate considers that many critical branches, edge cases, and error-handling paths remain untested, and in some cases, only one or two test cases are provided per function.

(iii) Do you believe there are faults that the test suite would miss? Justify your answers.

Yes, there are potential faults that the current test suite would likely miss due to its limited coverage and lack of testing for edge cases and potential errors.

In the Movie module, for instance, the get_number_of_breaks function could exhibit unintended behavior if the movie duration is less than 90 minutes and the genre is Fantasy. In this scenario, the calculation might result in a negative number of breaks (-1), which is both illogical and could lead to unexpected outcomes in subsequent functions relying on this value.

Similarly, in the Hall module, the expand_cinema function might encounter a fault when attempting to add a negative number of halls (number_of_halls_to_add) whose absolute value exceeds the current number of halls. This could lead to a negative

result in the loop that calculates the new hall number, specifically highest_hall_number + 1 + number_of_halls_to_add, potentially causing errors or undefined behavior in the system.

Q2.2 Use the instructions in README.md to obtain the current code coverage of the project. **Interpret and reflect on the metrics provided.** What impression does the code coverage metrics give you? Is it consistent with your own impression of the test suite?

The current code coverage of the project, as obtained using the instructions in the README.md, is reported to be 86%, which is significantly higher than the rough estimate of 70%.

This discrepancy highlights a common limitation of relying solely on code coverage metrics: while they provide a quantitative measure of the code executed during tests, they do not necessarily reflect the quality or completeness of the tests themselves.

The relatively high overall coverage percentage could be misleading, as it suggests that a significant portion of the code is being executed, but it doesn't account for untested scenarios, error handling, and boundary conditions that may introduce faults.

Stmts: the number of all statements
Miss: the number of missed statements
Branch: the number of all branches
BrPart: the number of missed branches
Cover: coverage

Coverage calculation:
Missed = (Miss + BrPart)
Total = (Stmts + Branch)
Coverage = (total-missed) / total

```
Name                    Stmts   Miss Branch BrPart  Cover
-------------------------------------------------------
src/__init__.py             0      0      0      0   100%
src/cinema.py              23      3     10      2    85%
src/hall.py                50      6     20      7    81%
src/movie.py               40      5     18      5    83%
src/utils.py                8      0      0      0   100%
test/__init__.py            0      0      0      0   100%
test/test_cinema.py        18      1      2      1    90%
test/test_hall.py          34      1      2      1    94%
test/test_movie.py         17      1      2      1    89%
-------------------------------------------------------
TOTAL                     190     17     54     17    86%
```

Q2.4 Now that you have performed your initial evaluation of the testing suite, it is time to improve the testing suite. Create and modify unit tests **using white-box techniques** with the goal of increasing the code coverage of the testing suite and improving the mutation score. Fix any faults you encounter in the source code whilst creating new tests.

Link to code: https://github.com/Crypthead/cinema_manager (by Stefan Van Heijningen)

Link to the solution with increased coverage:
https://github.com/alice-iceberg/CinemaManager

```
Name                    Stmts   Miss Branch BrPart  Cover
-------------------------------------------------------
src/__init__.py             0      0      0      0   100%
src/cinema.py              25      0     12      0   100%
src/hall.py                50      1     20      1    97%
src/movie.py               42      0     20      0   100%
src/utils.py                8      0      0      0   100%
test/__init__.py            0      0      0      0   100%
test/test_cinema.py        43      1     10      1    96%
test/test_hall.py          60      2     14      1    96%
test/test_movie.py         38      1      8      1    96%
-------------------------------------------------------
TOTAL                     266      5     84      4    97%
```